

Customer In-House Workshop

Christian Kurz [christian.kurz@nordicsemi.no]

Agenda

Day 1

- nRF Connect SDK: How Nordic uses Zephyr
- nRF54L Series
- nRF54L Series Trustzone

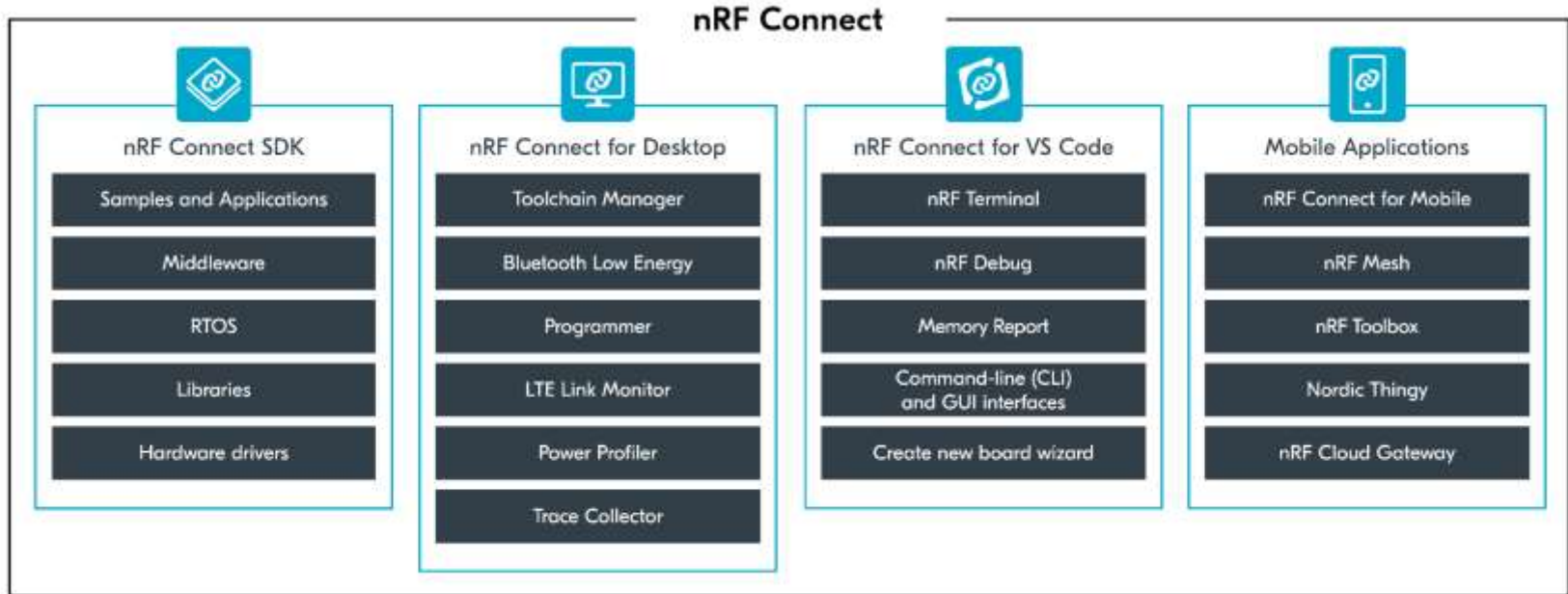
Day 2

- NCS/Zephyr's Power Management
- Creating a Custom Device Driver
- Bluetooth LE

nRF Connect SDK: How Nordic uses Zephyr

Development Tools

nRF Connect Platform Overview



PC Development Tools Overview



nRF Util (nrfutil)

- Powerful Command Line tool
- nRF Util replaces *nRF Command Line Tools* and *nrfjprog*



nRF Connect for Desktop

- Cross-platform tool framework for development applications
- It contains apps for testing, monitor, measure, optimize and programm your applications



nRF Connect for VS Code

- Extension for *Visual Studio Code*
- Provides easier, user-friendly GUI to build and configure your projects

Others

RTOS Debugging:

- Percepio
- Segger SystemView

Debugger:

- Segger Ozone

Other IDEs

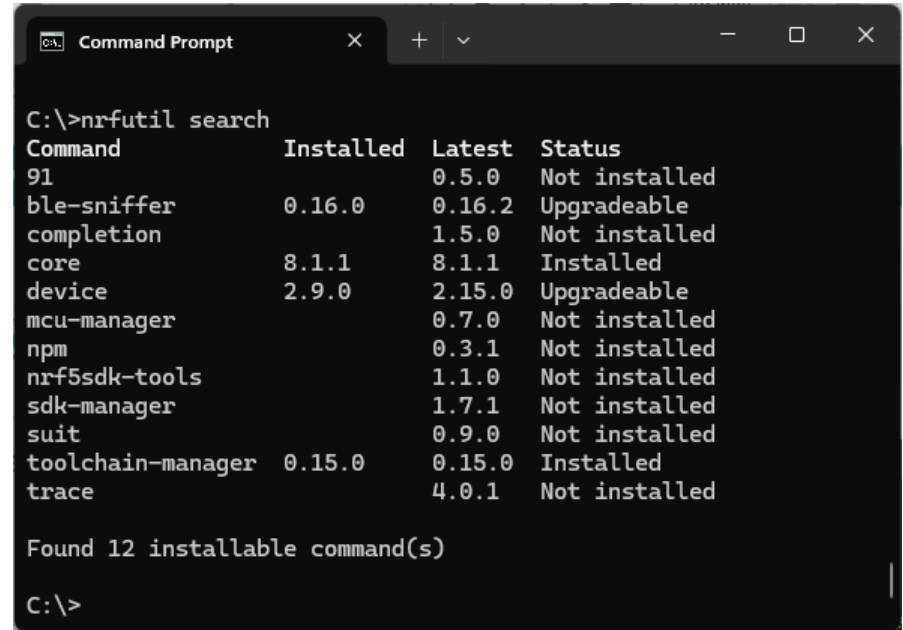
and more ...

nrfutil – Unified Command Line Utility

Further commands can be installed beside basic commands covered in *nrfutil*

Examples for installable *nrfutil* commands:

- Device
 - erase, program, verify, reset
 - readback protection, recover
 - memory read/write
- BLE-Sniffer
- MCU-Manager (serial transport only)
- SDK-Manager
 - nRF Connect SDK and toolchain installer



```
C:\>nrfutil search
```

Command	Installed	Latest	Status
91		0.5.0	Not installed
ble-sniffer	0.16.0	0.16.2	Upgradeable
completion		1.5.0	Not installed
core	8.1.1	8.1.1	Installed
device	2.9.0	2.15.0	Upgradeable
mcu-manager		0.7.0	Not installed
npm		0.3.1	Not installed
nrf5sdk-tools		1.1.0	Not installed
sdk-manager		1.7.1	Not installed
suit		0.9.0	Not installed
toolchain-manager	0.15.0	0.15.0	Installed
trace		4.0.1	Not installed

```
Found 12 installable command(s)
```

```
C:\>
```

AI-assisted development

Adding context and Nordic knowledge to generic AI tools



Develop source code, configure and build projects, flash and debug hardware, monitor logs and outputs.

Whatever IDE and LLM your team is using today: VS Code, Cursor, Claude Code, Windsurf, terminal, etc.

Access to all Nordic technical documentation, including nRF Connect SDK plus DevAcademy, and DevZone Q&A.
nRF Connect SDK prompt templates.
Field data from nRF Cloud.

nRF Connect for Desktop

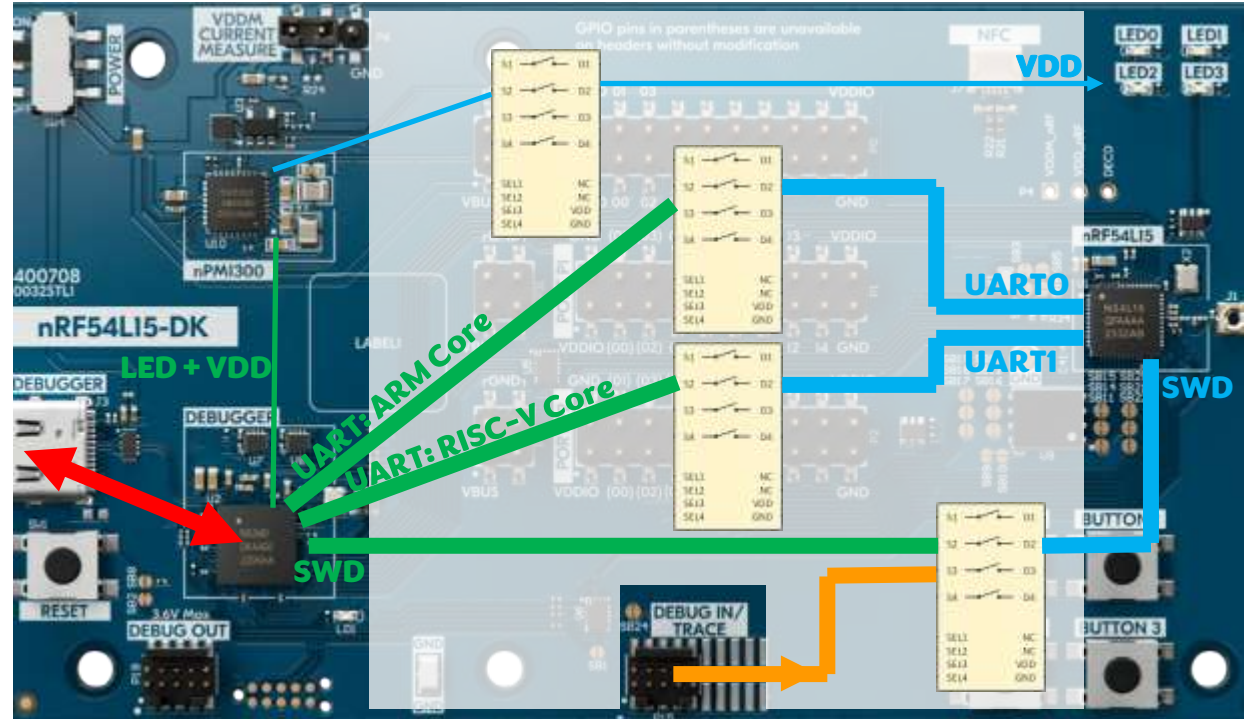
Tools aimed at SW development and general use:

- QuickStart for guided setup and install
- Programmer is a great visual tool for flashing Nordic devices, updating modem FW on nRF91 Series
- Power Profiler used to optimize power consumption
- Board Configurator for Dev Kit settings
- Serial Terminal – Nordics Terminal program with some advantages
- SoC specific Tools – Bluetooth, Matter, Cellular IoT, PMICs



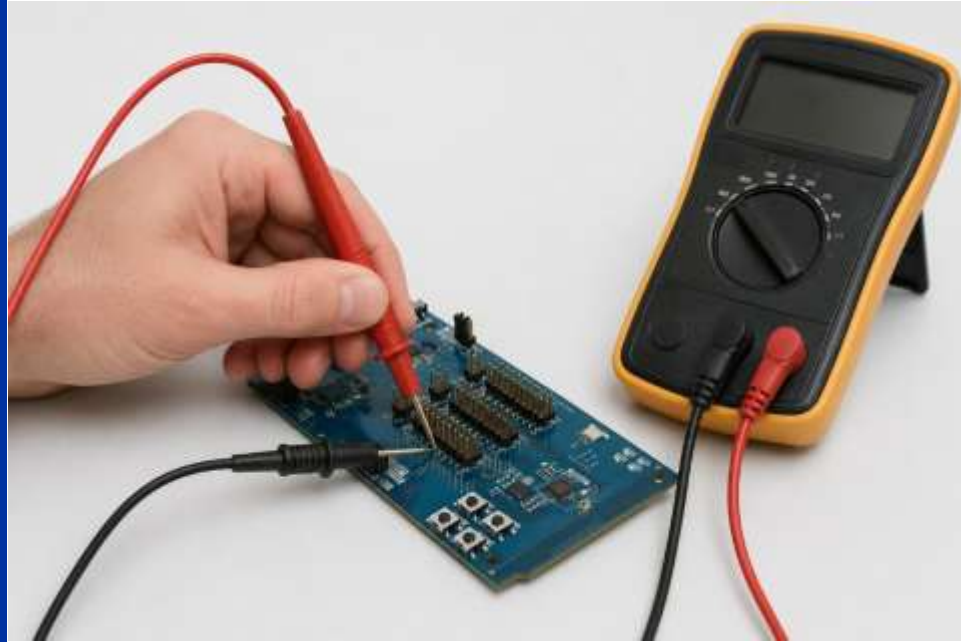
Digital Control of nRF54L15DK Board Settings

- VDD (nPM VOUT1)
 - VDD setting in 0.1 V steps (1.8 V to 3.3 V)
- Power to LEDs
- Connect port VCOM0 / Connect port VCOM1
 - Enable/disable UART Interface for Logging
- Software Debugger (SWD)
 - Debug on board or external debugger (DBG_IN)

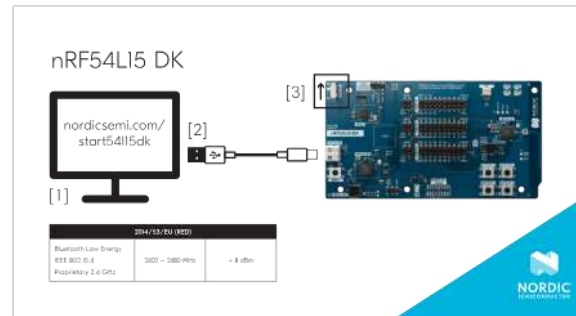
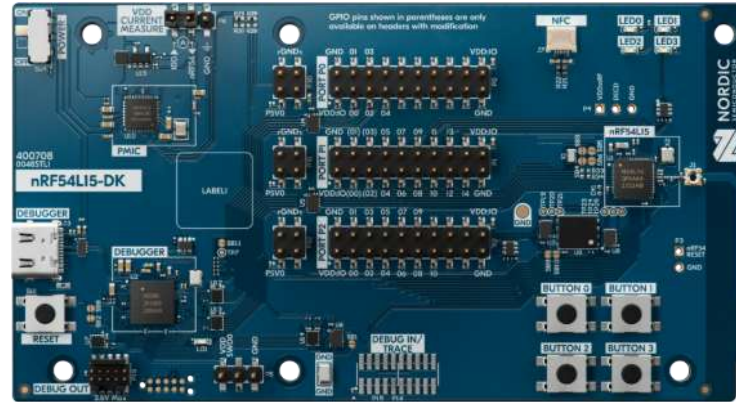


Hands-on

- Unboxing nRF54L15DK
- Board Configurator



Hands-on: Unboxing nRF54L15DK



Labels pointing to components on the nRF54L15-DK board:

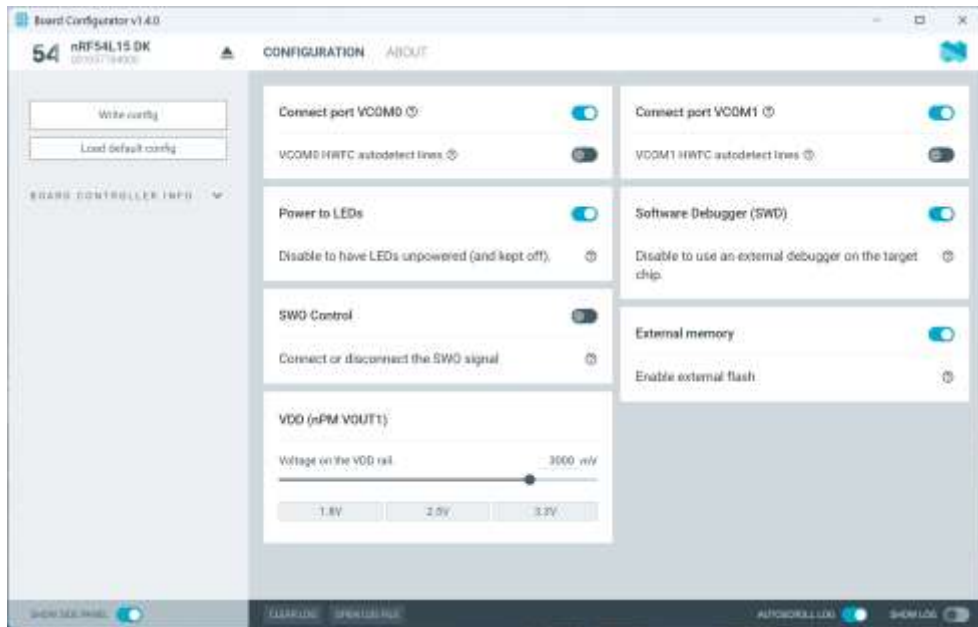
- Power switch
- nPMI300 PMIC
- Current measurement pins
- NFC antenna connector
- User-programmable LEDs
- SWF RF port for direct RF measurements
- 2.4 GHz PCB Antenna
- nRF54L15
- External memory
- User-programmable buttons
- Debug in
- SEGGER J-Link OB Programmer/debugger
- Debug out
- Reset button
- SEGGER J-LINK USB-C connector
- 40070B 0048STU
- nRF54L15-DK
- DEBUGGER
- DEBUG OUT
- DEBUG IN/TRACE
- DEBUGGER
- RESET
- SEGGER
- J-LINK
- USB-C
- connector
- RESET
- button
- SEGGER
- J-LINK
- OB
- Programmer/debugger
- Debug out
- Debug in
- User-programmable
- buttons
- External memory
- nRF54L15
- 2.4 GHz PCB
- Antenna
- SWF RF port
- for direct RF
- measurements
- User-programmable
- LEDs
- NFC antenna
- connector
- Current measurement
- pins
- nPMI300 PMIC
- Power switch

Step 1 - Check nRF54L15DK

1. Connect the nRF54L15DK to your PC using the USB cable. Verify that the board is powered by confirming that **LEDO** is blinking. If **LEDO** is not blinking, check the USB connection and ensure that the **POWER** switch is in the **ON** position before proceeding.

Step 2 - Connect nRF54L15DK to *Board Configurator*

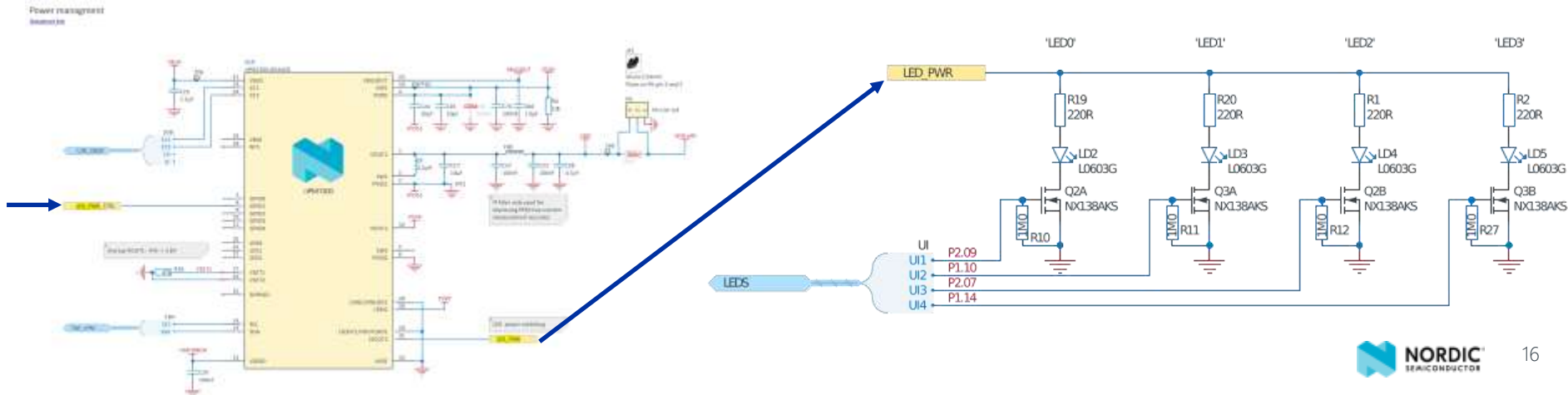
2. Start *nRF Connect for Desktop* and open the **Board Configurator** application.
3. Click **SELECT DEVICE**.
4. In the list of available devices, select your **nRF54L15DK** development kit. The application connects to the board.



Step 3 - Disable *Power to LEDs* and check what is happening on the DK

5. Disable **Power to LEDs**, then click **Write config**. Verify that LED0 stops blinking.
6. Power cycle the development kit by either:
Setting the **POWER** switch to **OFF** and then back to **ON**, or Disconnecting and reconnecting the USB cable.
7. After the board restarts, observe **LED0**. Is it blinking? Explain your observation.

💡 *Is the GPIO still toggling, or is only the LED power disabled?*





Step 4 - Enable *Power to LEDs*

8. In the **Board Configurator**, re-enable **Power to LEDs** and click **Write config**. Verify that **LED0** starts blinking again.

nRF Connect SDK

Options for Bluetooth LE Development

nRF Connect SDK

based on Zephyr RTOS



Fully featured Bluetooth LE stack

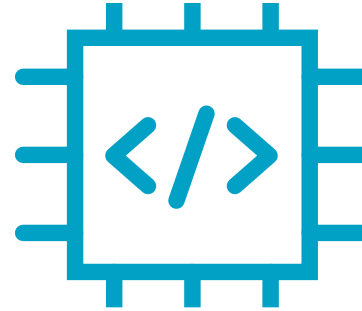
Extensive samples, libraries and modules

Multiprotocol wireless connectivity



nRF Connect SDK Bare-Metal Option

non-RTOS solution



Prebuilt Bluetooth LE stack (SoftDevice)

Samples and low-level drivers

Coexists with RTOS-based development

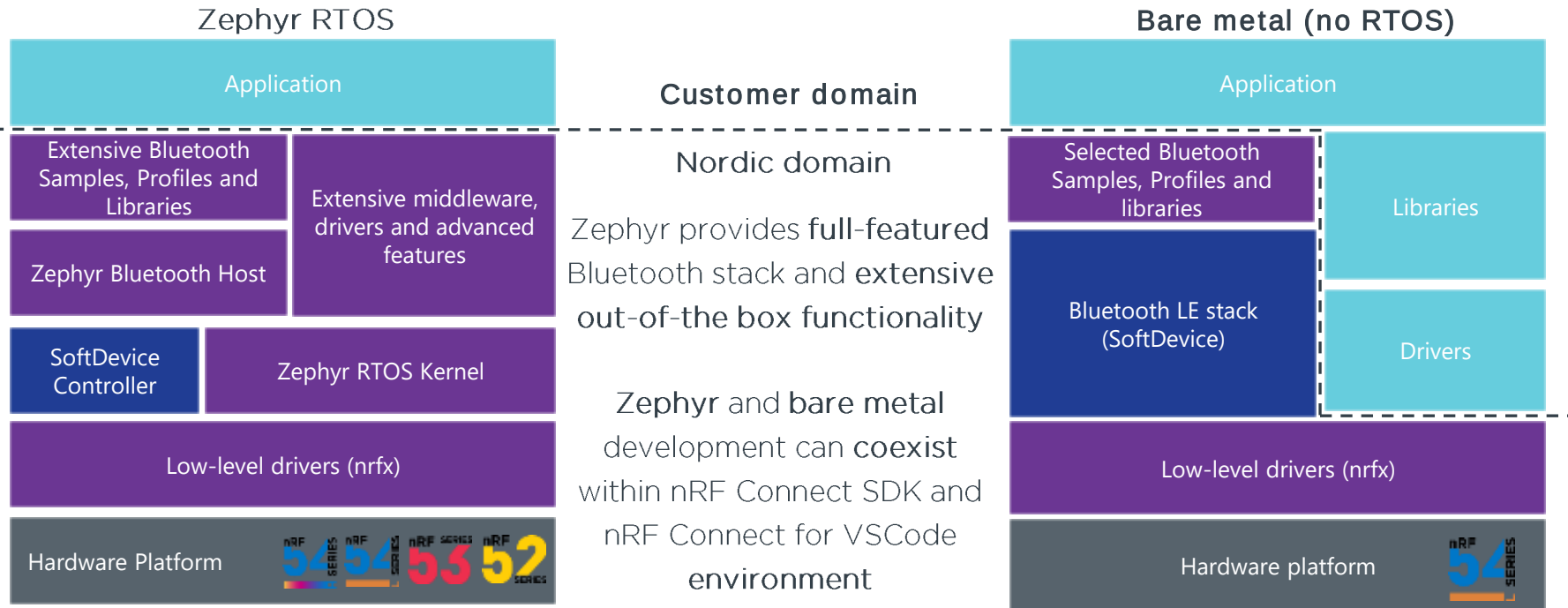


Common toolchain (west, GCC, Cmake, DeviceTree, KConfig) and development environment (VS Code)



nRF Connect SDK - Zephyr Vs Bare Metal

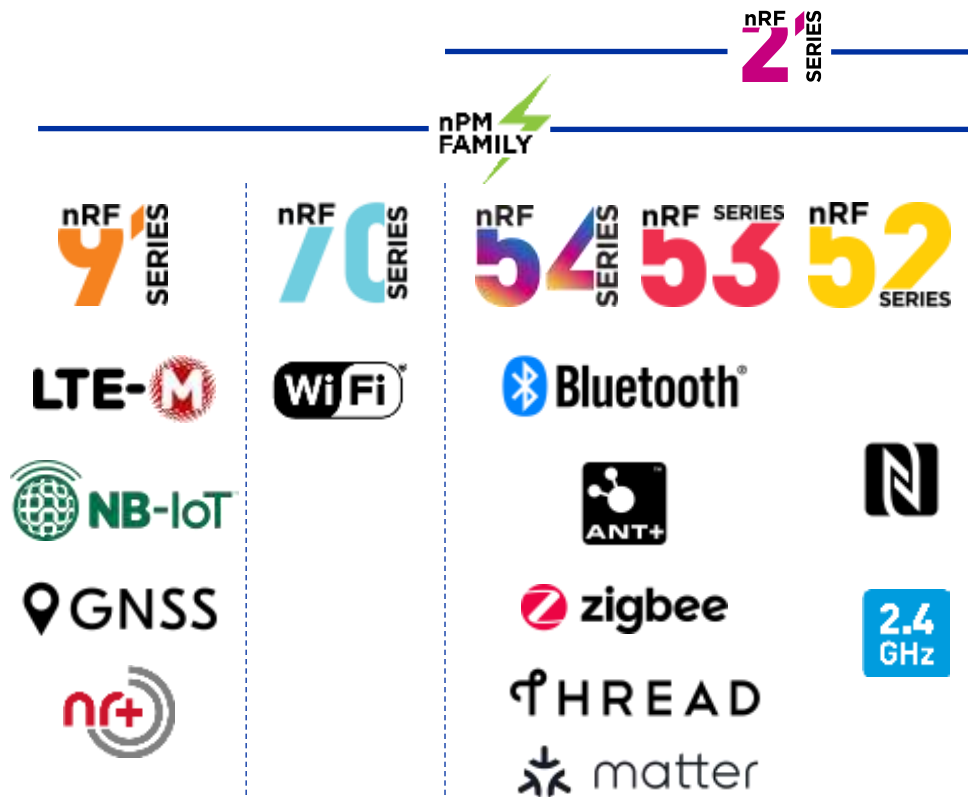
For Bluetooth LE development





nRF Connect SDK

- One Code Base and Toolchain for Nordic's Cellular IoT, Wi-Fi, Short-Range, and Companion Chips like PMiC and Front-End Modules
- Includes Bluetooth Low Energy, Bluetooth Mesh, Thread, Zigbee, Matter, ESB, Gazell, NFC, ANT, WiFi, LTE-M, NB-IoT, GNSS, DECT NR+



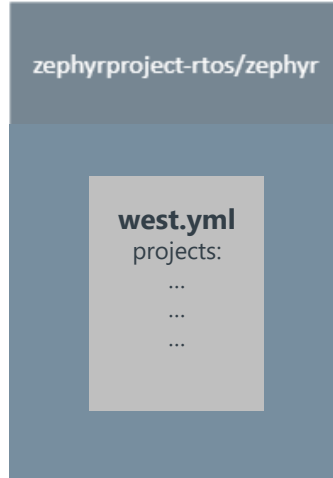
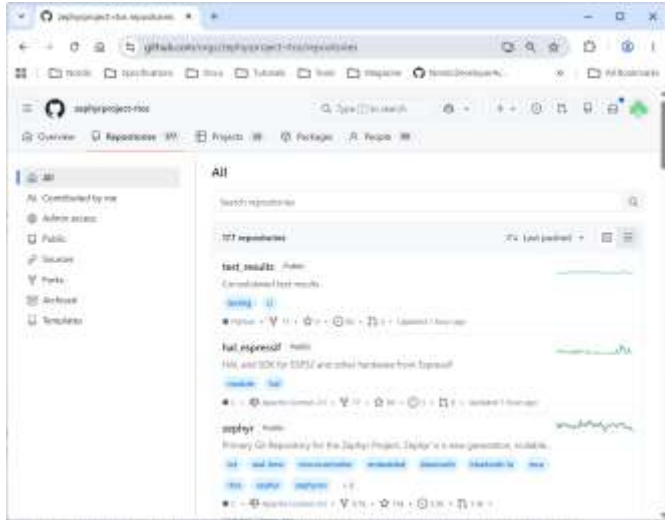
nRF Connect SDK Code Base

Code is organized in different repositories
([Nordic](#) and [Open Source](#) (OS) code):

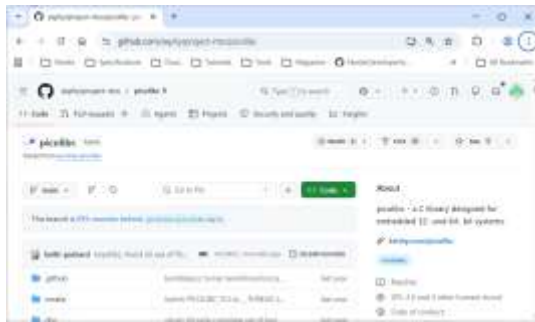
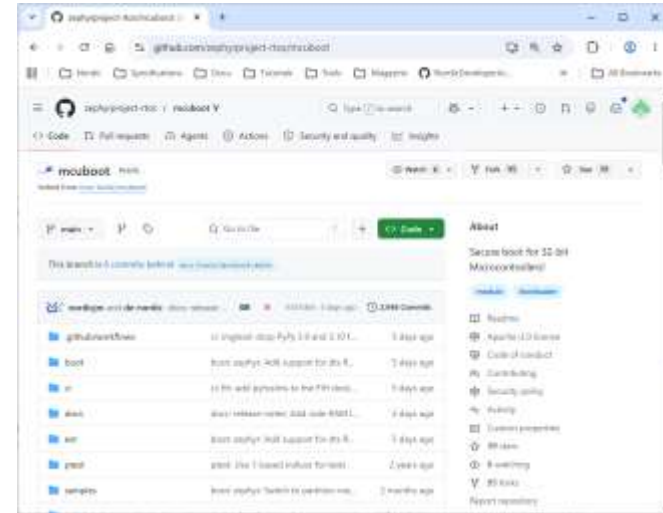
- [nrf](#) – Application & Connectivity Protocols
- [nrfxlib](#) – Compiled libraries
- [nrfx](#) – Peripheral Drivers
- [zephyr](#) – RTOS & Board Configuration (OS)
- [mcuboot](#) – Secure Bootloader (OS)
- Other Repositories:
 - Trusted Firmware-M, Matter, etc.



Zephyr Repositories (www.github.com/zephyrproject-rtos)

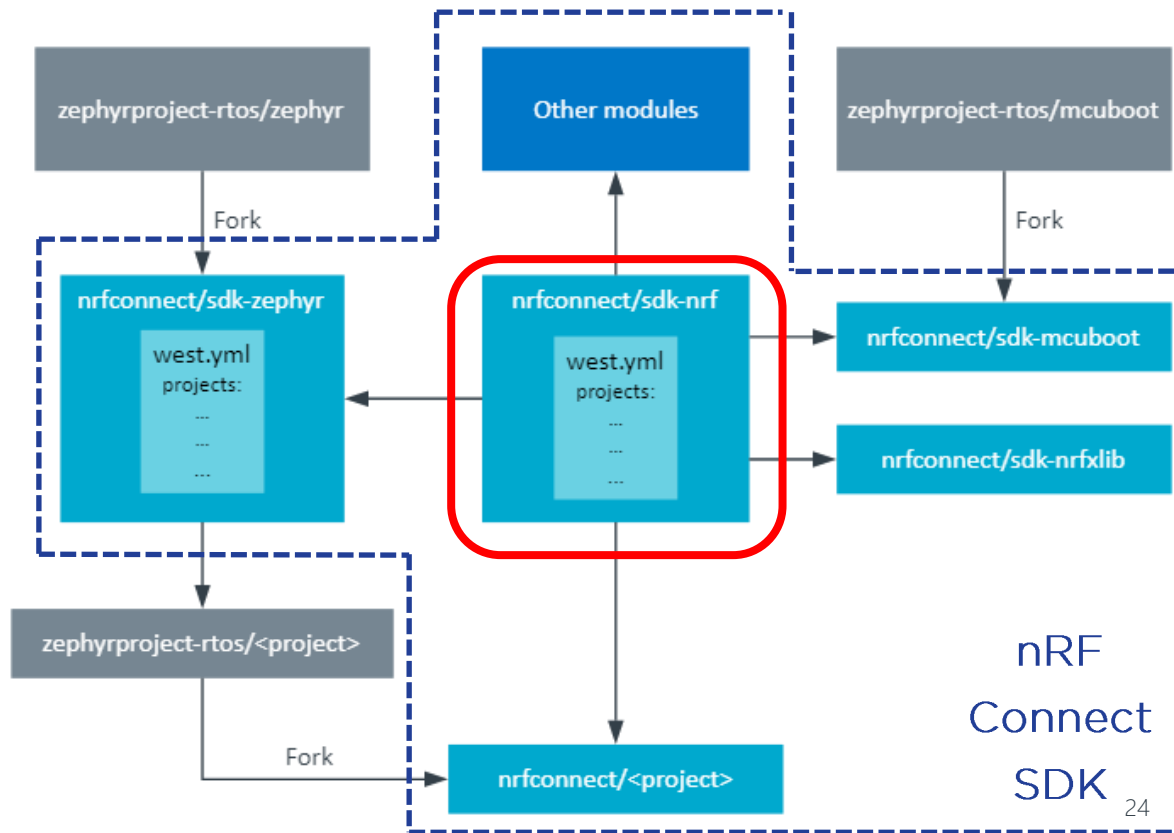


zephyrproject-rtos/mcuboot

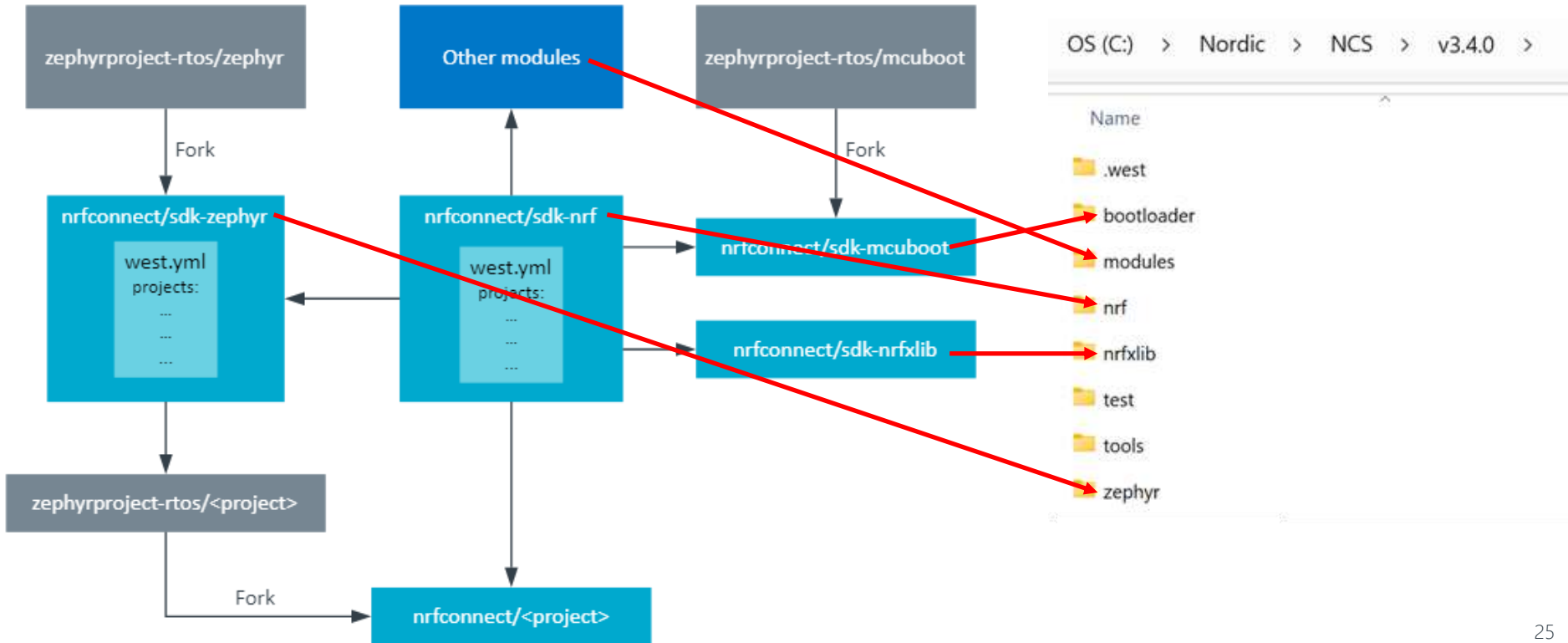


nRF Connect SDK Structure (www.github.com/nrfconnect)

- *nRF Connect SDK* is structured as star topology
→ **sdk-nrf** repo is the center
(github.com/nrfconnect/sdk-nrf)
- **sdk-nrf** repo contains manifest file **west.yml**
→ Lists and points to all other repos included in the SDK



Repository Structure vs. Installation Paths



Code Examples

Code Examples can be found in different repositories, e.g. *nrf* or *zephyr*.

nrf repository => sample folder

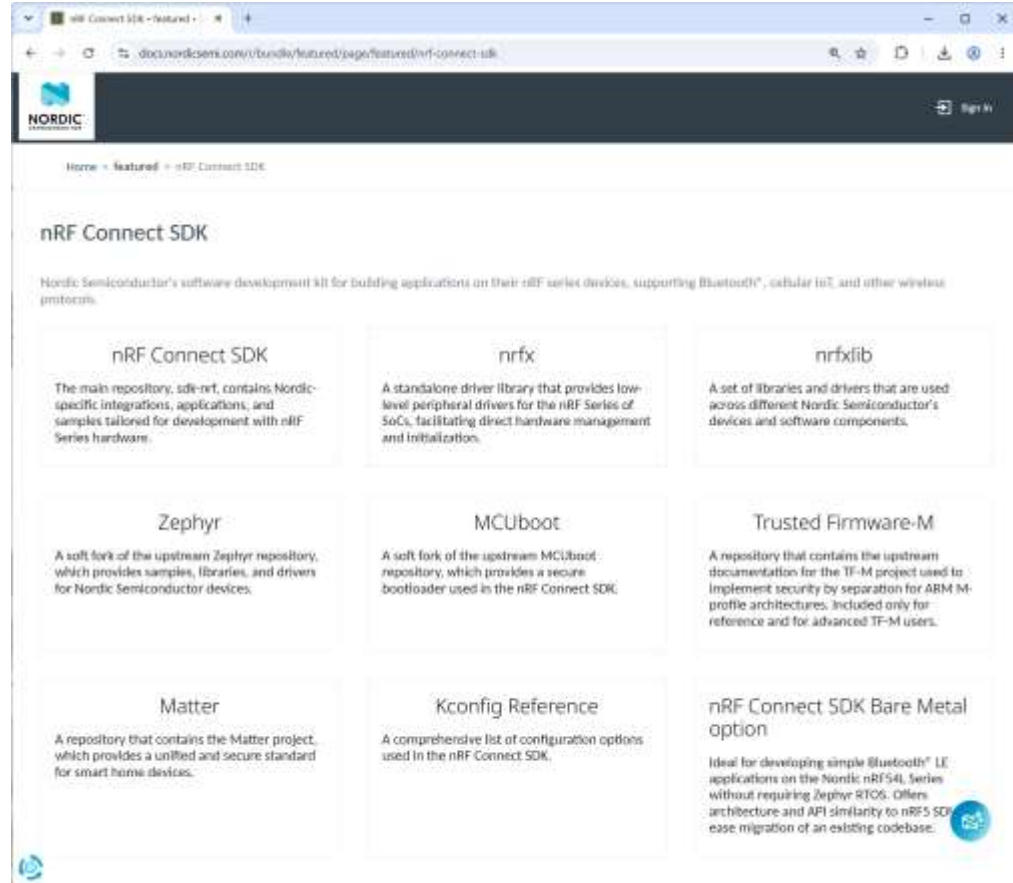
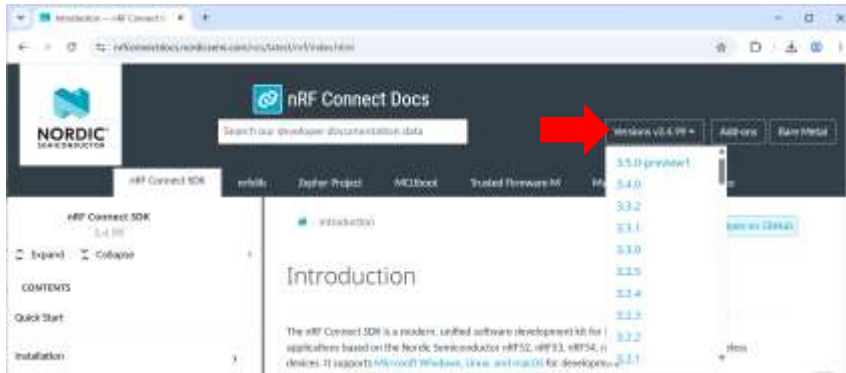
OS (C:) > Nordic > NCS > v3.4.0 > nrf > samples >		
Name	Date modified	Type
app_event_manager	7/2/2026 2:12 PM	File folder
app_event_manager_profiler_tracer	7/2/2026 2:12 PM	File folder
app_jwt	7/2/2026 2:12 PM	File folder
basic	7/2/2026 2:12 PM	File folder
benchmarks	7/2/2026 2:12 PM	File folder
bluetooth	7/2/2026 2:12 PM	File folder
bootloader	7/2/2026 2:12 PM	File folder
bootloader_deprecated_pm	7/2/2026 2:12 PM	File folder
caf	7/2/2026 2:12 PM	File folder
caf_sensor_manager	7/2/2026 2:12 PM	File folder
cellular	7/2/2026 2:12 PM	File folder
common	7/2/2026 2:12 PM	File folder
crypto	7/2/2026 2:12 PM	File folder

zephyr repository => sample folder

OS (C:) > Nordic > NCS > v3.4.0 > zephyr > samples >		
Name	Date modified	Type
application_development	7/2/2026 2:13 PM	File folder
arch	7/2/2026 2:13 PM	File folder
basic	7/2/2026 2:13 PM	File folder
bluetooth	7/2/2026 2:13 PM	File folder
boards	7/2/2026 2:13 PM	File folder
cpp	7/2/2026 2:13 PM	File folder
data_structures	7/2/2026 2:13 PM	File folder
drivers	7/2/2026 2:13 PM	File folder
hello_world	7/2/2026 2:13 PM	File folder
kernel	7/2/2026 2:13 PM	File folder
modules	7/2/2026 2:13 PM	File folder
net	7/2/2026 2:13 PM	File folder
philosophers	7/2/2026 2:13 PM	File folder
posix	7/2/2026 2:13 PM	File folder

nRF Connect SDK Documentation (docs.nordicsemi.com)

- Structured according to the repositories
- **Important:**
Select the SDK version you are using



Have you seen this button? What is it?

It's found on following web pages:

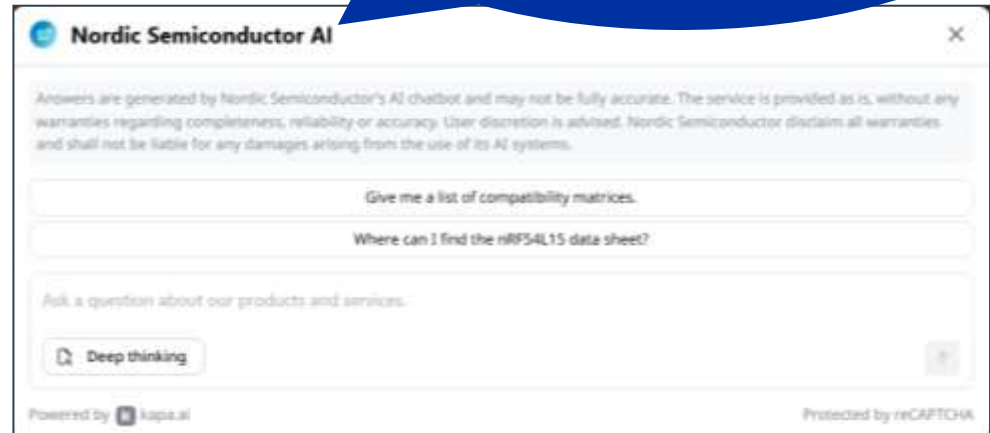
- [Nordic Semiconductor's TechDocs](#)
- [Nordic Semiconductor's DevZone](#)
- Within DevAcademy Courses



Nordic Semiconductor AI Agent

- Answering questions about Nordic Semiconductor's products, solutions, and documentation.
- AI Agent uses Nordic Semiconductor resources to look for an answer:
 - Official Nordic Semiconductor documentation
 - Nordic Semiconductor Blog posts
 - Nordic Semiconductor Forum Discussions
 - Nordic Semiconductor Support Resources
- Different Languages are supported

I am specialized in answering questions about Nordic Semiconductor's products, solutions, and documentation. If you have questions about Nordic products or technologies, I am happy to help!



Manage Source Code and Configurations

West
Multi-repository
management tool

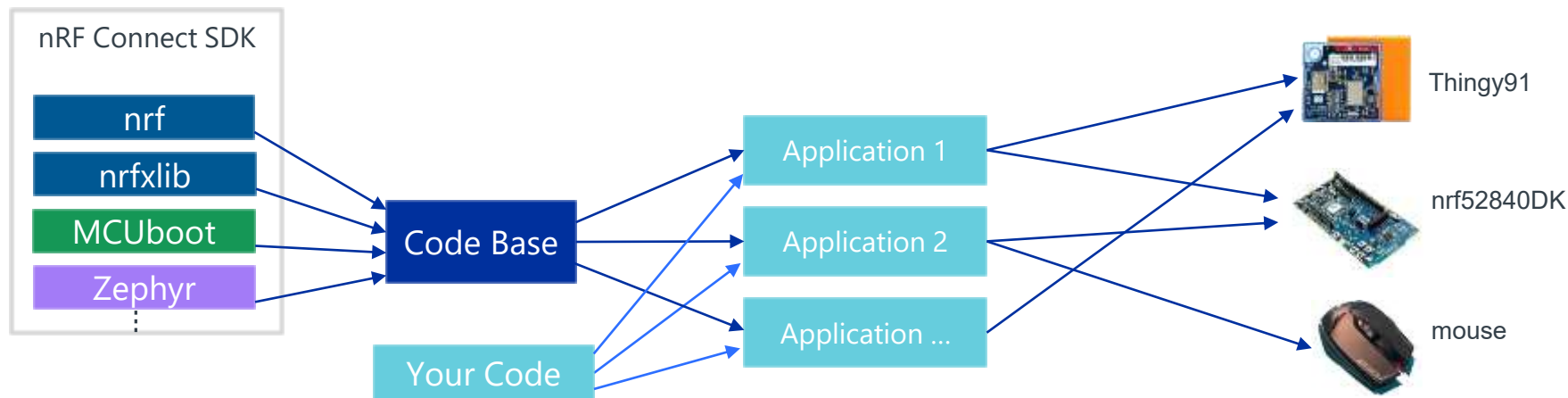
Kconfig
Source module / feature
configuration for compile

Device Tree
Target Board / Device
description

Clone / update
(Installation)

Configure features

Configure target



Zephyr Application Types

Repository

An app located in Zephyr repository is referred to as a *Zephyr Repository Application* (e.g. `hello_world` sample)

```
zephyrproject/  
├── .west/  
│   └── config  
└── zephyr/  
    ├── arch/  
    ├── boards/  
    ├── cmake/  
    ├── samples/  
    │   ├── hello_world/  
    │   └── ...  
    ├── tests/  
    └── ...
```

West Workspace

An app located within a workspace but outside the Zephyr repository is referred to as a *Zephyr Workspace Application*

```
zephyrproject/  
├── .west/  
│   └── config  
└── zephyr/  
    ├── bootloader/  
    ├── modules/  
    ├── tools/  
    ├── <vendor/private-repos>/  
    └── applications/  
        └── app/
```

nrf,
nrfxlib

NCS Add-ons:

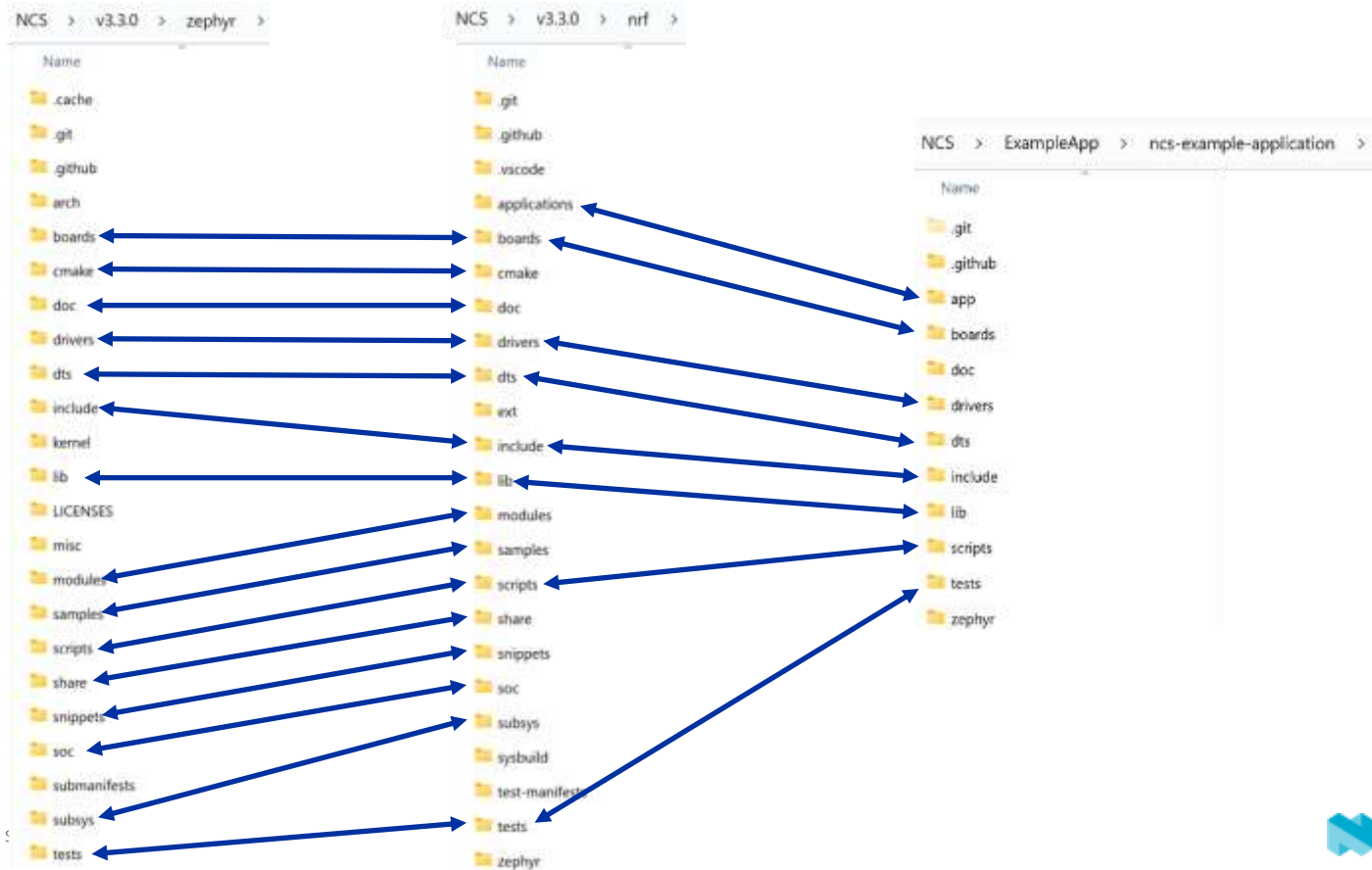
nRF Connect SDK example
application

Freestanding

Zephyr Freestanding Application is an app located outside a Zephyr workspace (e.g. nRF Connect SDK installation)

```
<home>/  
└── zephyrproject/  
    ├── .west/  
    │   └── config  
    ├── zephyr/  
    │   ├── bootloader/  
    │   ├── modules/  
    │   └── ...  
    └── app/  
        ├── CMakeLists.txt  
        ├── prj.conf  
        └── src/  
            └── main.c
```

zephyr, nrf, and other Folder Structure



Zephyr/NCS Application (minimal project)

```
Nordic
├── Workspace
│   └── MyProject
│       ├── CMakeLists.txt
│       ├── prj.conf
│       └── src
│           └── main.c
```

CMAKE

File: CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)

find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})

project(hello_world)

target_sources(app PRIVATE src/main.c)
```

KCONFIG

File: prj.conf

```
# nothing here! (default settings, e.g. CONFIG_GPIO=y)
```

Source Files

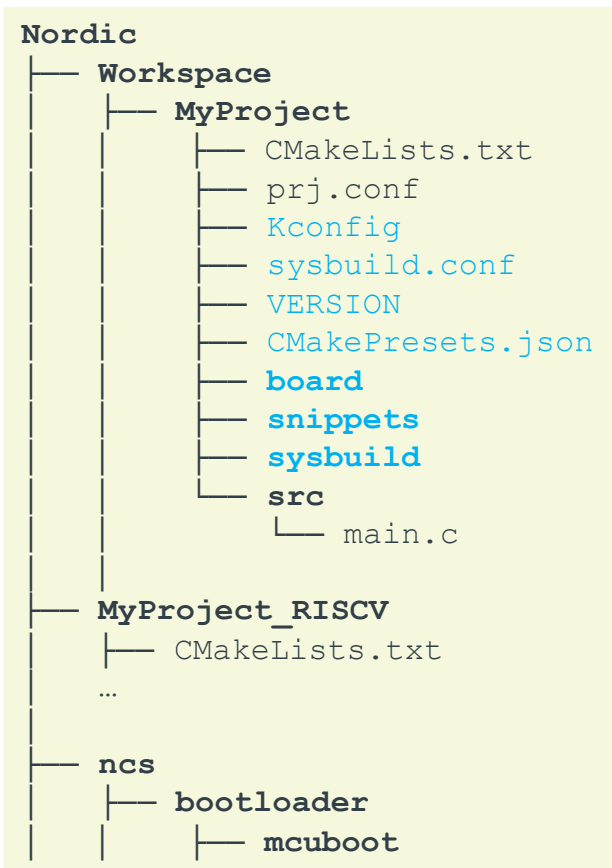
Application (user code) +
selected software modules from
Code Base (SDK)

File: src/main.c

```
#include <zephyr/kernel.h>

int main(void) {
    printk("Hello World!\n");
    return 0;
}
```

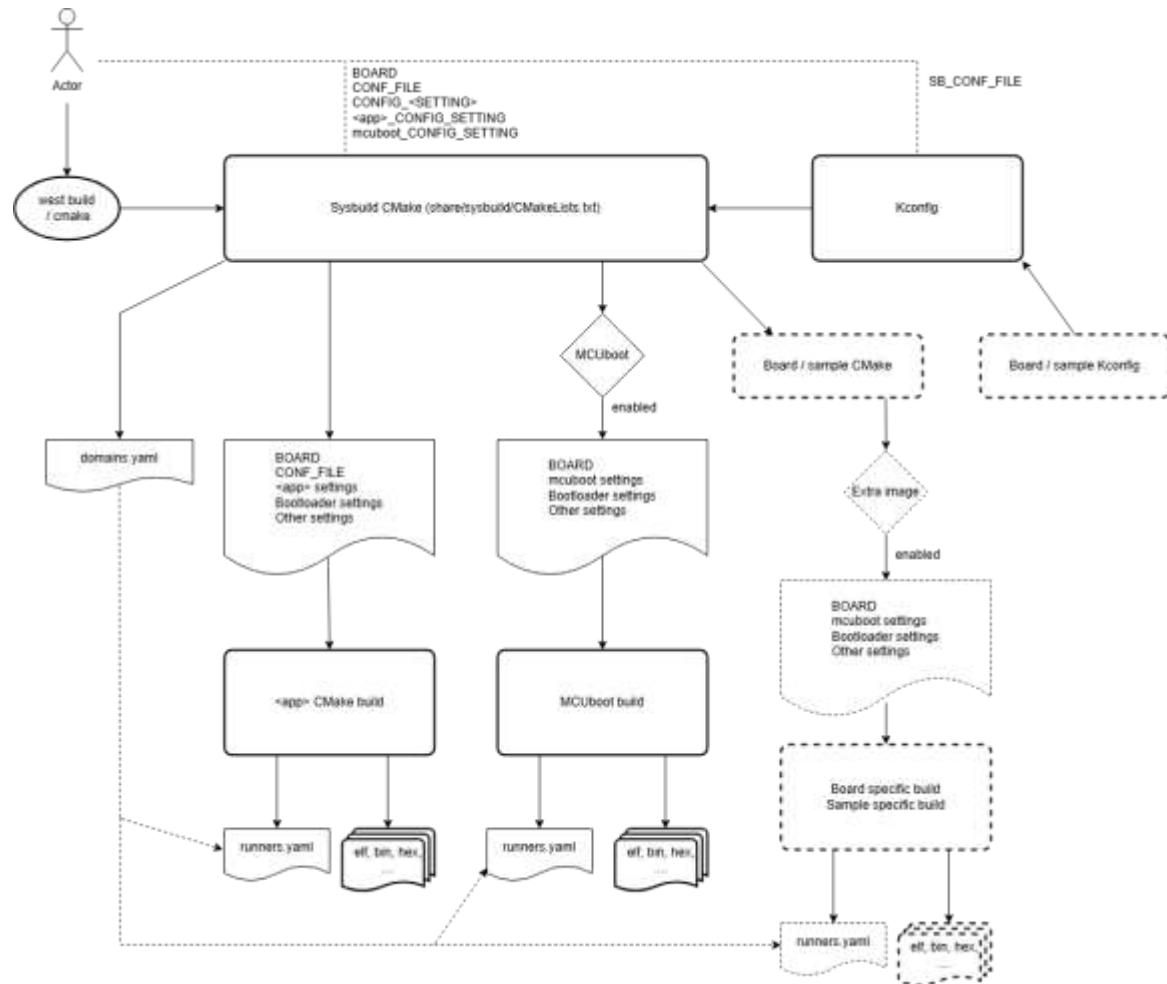
Zephyr/NCS Application – optional parts



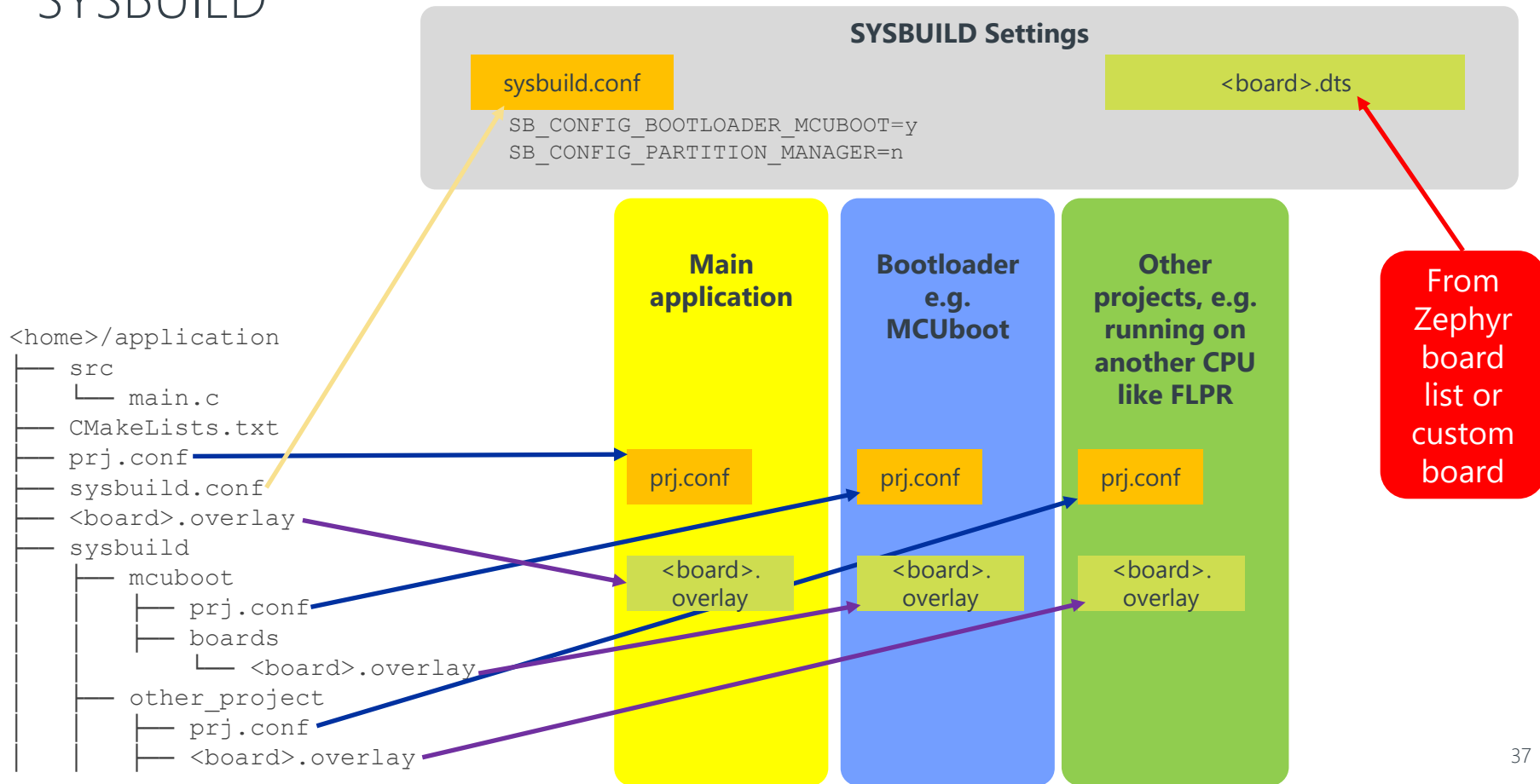
- **Kconfig**
User-defined Kconfig symbols for custom application
- **VERSION**
Application Version Management. Version is accessible from Source Code, Kconfig, Cmake, and Shell.
- **CMakePresets.json**
Stores build configuration settings for your application.
- **Board folder**
Overlay DTS file overwrites pre-defined settings. Or an own board-definition
- **Snippets folder**
common configurations (Kconfig and DeviceTree)
- **Sysbuild.conf + sysbuild folder**
used to combine multiple project builds together

SYSBUILD

SYSBUILD

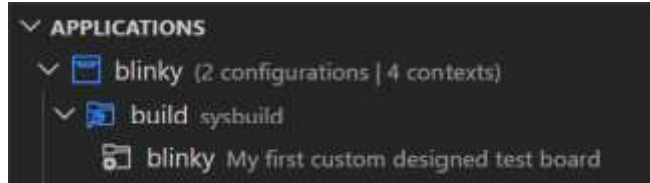


SYSBUILD

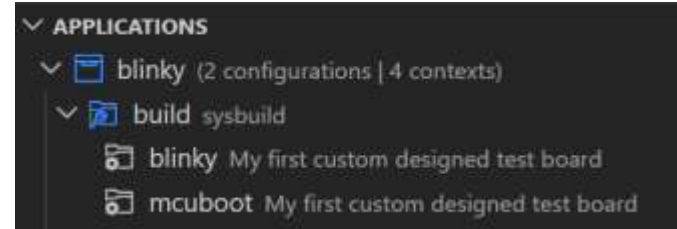


Sysbuild - How does this look like in VS Code?

Single Project Application using SYSBUILD



Multi-Project Application (including MCUboot)



Select SYSBUILD or one of the other Projects

